

**PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR SEDE MANABÍ CAMPUS**

**PORTOVIEJO**

**CARRERA DE SOFTWARE**

**MATERIA**

**GESTIÓN PARA LA VERIFICACIÓN Y VALIDACIÓN DEL SOFTWARE**

**ESTUDIANTES**

John Steven Lopez Velez

Luiggi Andrei Bernardi Zambrano

Santiago Esquetini

**DOCENTE**

Ing. Jose Naranjo

**FECHA**

**29 de Mayo de 2026**

**PAO 2026-01**



## Definiciones

- **P2P estructurado:** los nodos siguen una topología lógica determinista (anillo, árbol XOR) que asigna cada recurso a un nodo calculable y permite localizarlo en  $O(\log N)$  saltos.
- **P2P no estructurado:** no hay topología impuesta; los nodos se conectan al azar y las búsquedas se propagan por inundación, paseo aleatorio o heurísticas.
- **DHT (Distributed Hash Table):** tabla hash repartida entre los nodos. Cada clave  $k$  (típicamente el hash SHA-1/256 de un recurso) se asigna al nodo cuyo ID está “más cerca” según una métrica.
- **Chord (MIT, 2001):** DHT sobre un anillo módulo  $2^m$ . Cada nodo mantiene una finger table con  $\log_2 N$  atajos exponenciales y un proceso `stabilize()` repara la estructura cuando entran o salen nodos (churn).
- **Kademlia (NYU, 2002):** DHT con distancia XOR simétrica; cada nodo guarda 160  $k$ -buckets de contactos por proximidad y los lookups consultan  $\alpha=3$  nodos en paralelo. Base de BitTorrent, Ethereum, I2P e IPFS.
- **IPFS (Protocol Labs, 2014):** sistema de ficheros distribuido que direcciona por contenido. El archivo se trocea, se monta un Merkle DAG y el CID raíz (un hash) lo identifica. Kademlia descubre quién lo aloja; Bitswap lo intercambia.
- **Gnutella (Nullsoft, 2000):** P2P no estructurado puro; busca por flooding con  $TTL=7$ . La versión 0.6 introduce ultrapeers (nodos potentes con mini-índice) para mitigar el coste  $O(N)$ .
- **Freenet (Clarke, 2000):** P2P no estructurado anónimo. Enrutamiento heurístico por similitud de claves, almacenamiento cifrado y caché replicada en cada salto que protege al originador.

## Ideas clave

- **Enrutamiento DHT:** cada salto reduce a la mitad la distancia  $\rightarrow \leq \log_2 N$  saltos conociendo solo  $\log N$  vecinos (búsqueda binaria distribuida).
- **Chord vs Kademlia:** Chord es asimétrico y necesita `stabilize()`; Kademlia es simétrico y refresca sus  $k$ -buckets con el tráfico normal  $\rightarrow$  mejor frente al churn.
- **Replicación:** cada clave se almacena en los  $k$  nodos más cercanos ( $k \approx 20$ ) para sobrevivir a fallos sin coordinador.
- **IPFS = CID + DHT + Bitswap:** el CID identifica el contenido por su hash (no por su ubicación), la DHT publica “yo tengo este CID” y Bitswap intercambia los bloques.
- **Mutabilidad en IPFS:** IPNS y DNSLink añaden nombres estables sobre CIDs inmutables a cambio de confiar en firma o DNS y de más latencia.

## Tabla comparativa

| Sistema  | Tipo            | Topología        | Búsqueda               | Coste                  |
|----------|-----------------|------------------|------------------------|------------------------|
| Chord    | Estructurado    | Anillo $m$ -bits | Finger table           | $O(\log N)$            |
| Kademlia | Estructurado    | Árbol XOR        | $k$ -buckets paralelos | $O(\log N)$            |
| IPFS     | Híbrido         | Merkle DAG + DHT | Provider records       | $O(\log N)$            |
| Gnutella | No estructurado | Malla aleatoria  | Flooding + TTL         | $O(N)$                 |
| Freenet  | No estructurado | Malla heurística | Routing por clave      | $\sim O(\log N)$ prob. |



### **Preguntas**

1. Cuando un k-bucket está lleno y llega un contacto nuevo, ¿a quién conserva la red y por qué eso la hace resistente al churn?
2. Tener el CID de un archivo, ¿garantiza poder descargarlo siempre? ¿Qué asegura el CID, qué no, y cómo se consigue lo que falta?